Classificazione di esercizi in palestra con IMU

Jacopo Ciotti, Gian Marco Tuzi, Alessandro Papa

Università Politecnica delle Marche

20 gennaio 2026

Obiettivo

Realizzazione di un modello per la classificazione delle attività (esercizi in palestra) basandosi su dati acquisiti da sensore inerziale indossato sull'avambraccio (accelerometro e giroscopio). Materiale:

- Paper MyoGym: https://dl.acm.org/doi/epdf/10.1145/3123024.3124400.
- Dataset MyoGym: https://github.com/lok63/HAR-for-fitness-data/blob/master/data/MyoGym/MyoGym.mat.
- Codice python (Colab): https://colab.research.google.com/drive/1Wv4LO2ez0ALIBgigdJV3NZuezZ_dKSkm?usp=sharing.

Workflow

1. Analisi del dataset MyoGym e delle feature IMU fornite.
2. Addestramento del modello con tecnica `LOSO`.
3. Selezione delle classi di interesse e accorpamento degli esercizi simili.
4. Ri-addestramento del modello usando il nuovo dataset.
5. Analisi dei risultati soggetto per soggetto.
6. Addestramento del modello con split 80/20.
7. Feature Extraction.
8. Addestramento del modello usando le feature calcolate.
9. Confronto dei risultati ottenuti tra feature fornite e feature calcolate.
10. Confronto tra modelli tramite Classification Learner.
11. Analisi della rilevanza delle feature tramite `ANOVA`.

Dataset MyoGym

- I dati sono stati raccolti utilizzando un Myo Armband. Il Myo possiede 8 sensori EMG e un'IMU a nove assi contenente un giroscopio a tre assi, un accelerometro a tre assi e un magnetometro a tre assi (i dati del magnetometro non sono disponibili nel dataset). Nello studio, il Myo è stato posizionato sull'avambraccio destro, in modo che l'IMU fosse posizionato sulla parte superiore dell'avambraccio, mentre gli 8 sensori EMG erano distribuiti uniformemente attorno al braccio. In questo studio, per la raccolta dati è stata utilizzata la frequenza di 50 Hz.

Dataset MyoGym (cont.)

Il dataset `MyoGym.mat` contiene:

- `raw_data`:
  - colonne 1, 10 e 14: timestamps (ms).
  - colonne 2-9: segnali EMG.
  - colonne 11-13: segnali dell'accelerometro.
  - colonne 15-17: segnali del giroscopio.
- `raw_data_labels`:
  - colonna 1: label degli esercizi 1-30, 99.
  - colonna 2: label del soggetto 1-10.
- `features_acc` e `features_gyro`:
  - colonne 1-57: features per accelerazione e giroscopio (slide 7).
- `features_labels`:
  - colonna 1: label degli esercizi 1-30, 99.
  - colonna 2: label del soggetto 1-10.

Exercise Classes

Class	Name	Muscle group	Posture	One-arm, both or alternate
1	Seated Cable Rows	Middle Back	Seated	Both
2	One-Arm Dumbbell Row	Middle Back	Bent Over	One-arm
3	Wide-Grip Pulldown Behind The Neck	Lats	Seated	Both
4	Bent Over Barbell Row	Middle Back	Bent Over	Both
5	Reverse Grip Bent-Over Row	Middle Back	Bent Over	Both
6	Wide-Grip Front Pulldown	Lats	Seated	Both
7	Bench Press	Chest	On back	Both
8	Incline Dumbbell Flyes	Chest	Seated inclined	Both
9	Incline Dumbbell Press	Chest	Seated inclined	Both
10	Dumbbell Flyes	Chest	On back	Both
11	Pushups	Chest	On hands & toes/-knees	Both
12	Leverage Chest Press	Chest	Seated	Both
13	Close-Grip Barbell Bench Press	Triceps	On back	Both
14	Bar Skullcrusher	Triceps	On back	Both
15	Triceps Pushdown	Triceps	Standing	Both
16	Bench Dip / Dip	Triceps	Weight on hands	Both
17	Overhead Triceps Extension	Triceps	Standing	Both
18	Tricep Dumbbell Kickback	Triceps	Bent over	One-arm
19	Spider Curl	Biceps	Seated	Both
20	Dumbbell Alternate Bicep Curl	Biceps	Standing	Alternate l
21	Incline Hammer Curl	Biceps	Seated inclined	Both
22	Concentration Curl	Biceps	Seated	One-arm
23	Cable Curl	Biceps	Standing	Both
24	Hammer Curl	Biceps	Standing	Alternate
25	Upright Barbell Row	Shoulders	Standing	Both
26	Side Lateral Raise	Shoulders	Standing	Both
27	Front Dumbbell Raise	Shoulders	Standing	Alternate
28	Seated Dumbbell Shoulder Press	Shoulders	Seated	Both
29	Car Drivers	Shoulders	Standing	Both
30	Lying Rear Delt Raise	Shoulders	On stomach	Both
99	NULL			

Features

```
X = (abs(fft(x_accel, 200)));
Y = (abs(fft(y_accel, 200)));
Z = (abs(fft(z_accel, 200)));
X = (X - mean(X))/std(X);
Y = (Y - mean(Y))/std(Y);
Z = (Z - mean(Z))/std(Z);
```

```
1  HjorthParameters(x_accel);
2  HjorthParameters(y_accel);
3  HjorthParameters(z_accel);
4  std(x_accel);
5  std(y_accel);
6  std(z_accel);
7  mean(x_accel);
8  mean(y_accel);
9  mean(z_accel);
10 min(x_accel);
11 min(y_accel);
12 min(z_accel);
13 max(x_accel);
14 max(y_accel)
15 max(z_accel);
16 median(x_accel);
17 median(y_accel);
18 median(z_accel);
19 prctile(x_accel, 25);
20 prctile(y_accel, 25);
21 prctile(z_accel, 25);
22 prctile(x_accel, 75);
23 prctile(y_accel, 75);
24 prctile(z_accel, 75);
25 prctile(x_accel, 10);
26 prctile(y_accel, 10);
27 prctile(z_accel, 10);
28 prctile(x_accel, 90);
29 prctile(y_accel, 90);
30 prctile(z_accel, 90);
31 prctile(x_accel, 95);
32 prctile(y_accel, 95);
33 prctile(z_accel, 95);
34 prctile(x_accel, 5);
35 prctile(y_accel, 5);
36 prctile(z_accel, 5);
37 corr(x_accel, y_accel);
38 corr(x_accel, z_accel);
39 corr(y_accel, z_accel);
40 sum(((x_accel(2:end)>=0) &
   (x_accel(1:end-1)<=0)));
41 sum(((y_accel(2:end)>=0) &
   (y_accel(1:end-1)<=0)));
42 sum(((z_accel(2:end)>=0) &
   (z_accel(1:end-1)<=0)));
43 sum(((x_accel(2:end)>=mean(
   x_accel)& (x_accel(1:end-1)
   <=mean(x_accel)));
44 sum(((y_accel(2:end)>=mean(
   y_accel)& (y_accel(1:end-1)
   <=mean(y_accel)));
45 sum(((z_accel(2:end)>=mean(
   z_accel)& (z_accel(1:end-1)
   <=mean(z_accel)));
46 sum(X(1));
47 sum(Y(1));
48 sum(Z(1));
49 sum(X(1:5));
50 sum(Y(1:5));
51 sum(Z(1:5));
52 sum(X(6:10));
53 sum(Y(6:10));
54 sum(Z(6:10));
55 entropy(x_accel);
56 entropy(y_accel);
57 entropy(z_accel);
```

Le feature sono 57 per l'accelerometro e 57 per il giroscopio, per un totale di **114**.

Primo addestramento

- Per l'addestramento del modello sono state utilizzate le feature fornite dal Dataset `MyoGym.mat`.
- Sono state considerate tutte le classi da 1 a 30, escludendo la classe 99 (`NULL`).
- Il modello utilizzato è un classificatore `SVC` (Support Vector Classifier), configurato con i seguenti parametri:
  - `kernel="rbf"`, consente di modellare relazioni non lineari tra le feature.
  - `C=1.0`, bilancia la regolarizzazione e gli errori di classificazione.
  - `gamma="scale"`, imposta automaticamente l'influenza dei singoli campioni.
  - `class_weight="balanced"`, assegna pesi inversamente proporzionali alla frequenza delle classi per gestire eventuali sbilanciamenti nel dataset.
- Come metodo di cross-validazione è stata utilizzata la tecnica `LOSO` (Leave-One-Subject-Out).

Primo addestramento (cont.)

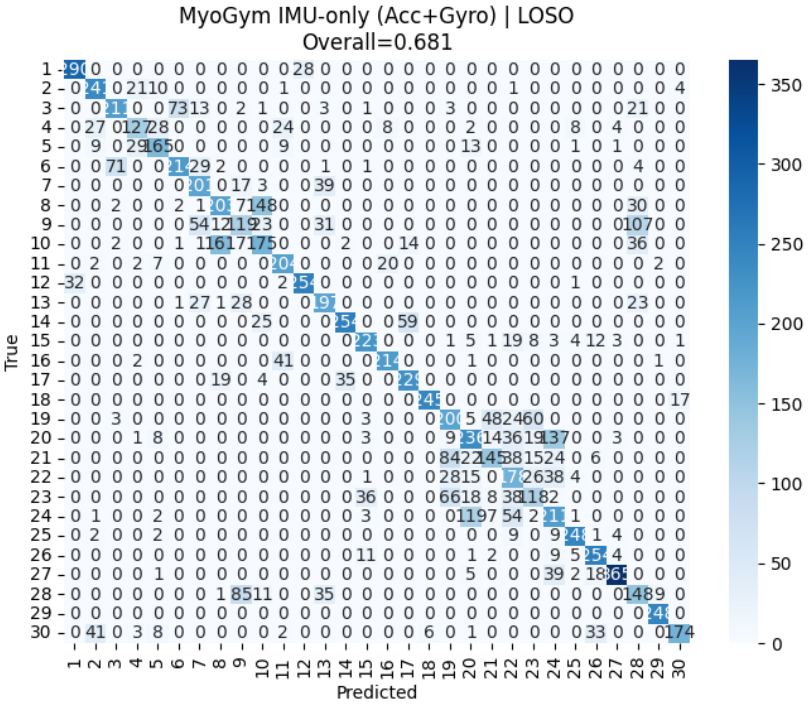

Figura 1: Matrice di confusione con tutte le classi (1-30)

```
accuracy = 0.68
```

Selezione classi

- Con soli dati da accelerometro e giroscopio non tutte le classi sono distinguibili in modo netto; per questo motivo sono state selezionate solo le feature di alcune classi in base a `f1-score` ≥ 0.75 e a valori elevati di `precision` e `recall`.
- In particolare, le varianti di `Biceps Curl` (classi 19-24) non sono singolarmente distinguibili senza EMG. Invece di eliminarle, tutte le varianti sono state accorpate in una classe 31.
- Questo nuovo set di dati è stato utilizzato per addestrare lo stesso modello `SVC` sempre con tecnica `LOSO`.

Classi finali

```
class_keep = [1, 6, 7, 11, 12, 14, 15, 16, 17, 18, 25, 26,
27, 29, 30, 31]
```

Secondo addestramento

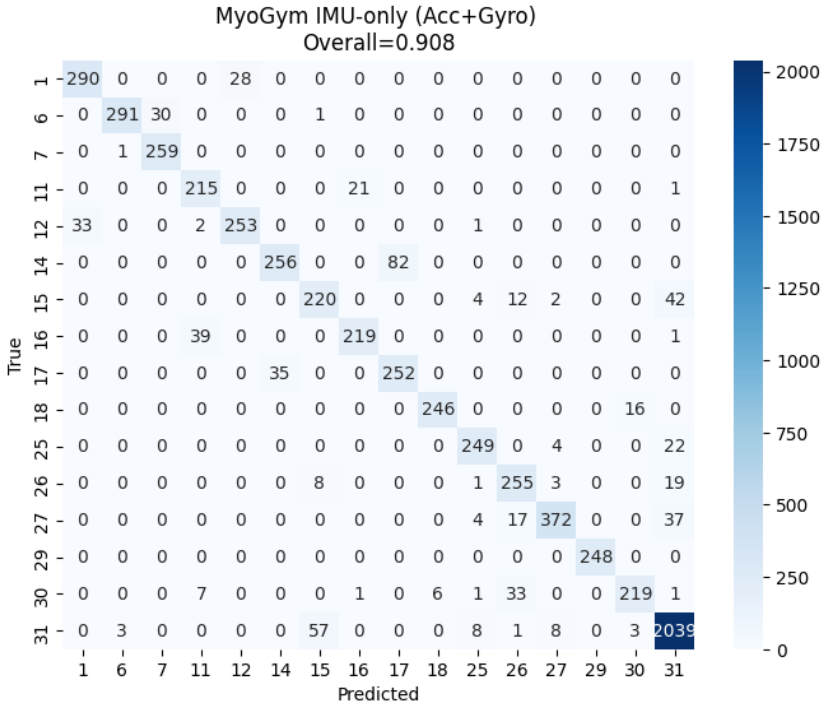

Figura 2: Matrice di confusione con solo le classi selezionate

```
accuracy = 0.91
```

Analisi per soggetto (LOSO)

- L'accuracy globale può mascherare prestazioni non uniformi: ogni soggetto, infatti, presenta uno stile motorio differente, quindi la stessa classe di esercizio non genera lo stesso segnale per tutti.
- L'analisi per soggetto consente di valutare la robustezza e la capacità di generalizzazione del modello, individuando soggetti "sfortunati" (fuori distribuzione), stili di movimento atipici o segnali IMU meno informativi.
- Come si può notare dalla Tabella 1, i soggetti 1 e 6 sono effettivamente più "sfortunati" degli altri. Essi hanno, infatti, un'accuratezza inferiore, il che conferma che non tutte le persone generano gli stessi segnali per i medesimi movimenti.

Analisi per soggetto (cont.)

Subject	$n_{\textbf{test}}$	**Accuracy**
1	722	0.6288
6	667	0.7916
7	656	0.9116
3	567	0.9400
10	501	0.9401
5	674	0.9718
2	817	0.9718
8	589	0.9830
9	773	0.9884
4	512	0.9902

Tabella 1: Accuratezza per soggetto (`LOSO`)

Split 80/20

- È stata effettuata un'ulteriore analisi intra-soggetto accorpando le feature di tutti i soggetti e suddividendo il nuovo set di dati risultante secondo lo split 80/20.
- In questo scenario l'informazione sul soggetto che esegue l'esercizio non viene considerata, favorendo invece l'apprendimento di pattern motori condivisi.
- Lo split 80/20 permette di valutare le prestazioni del modello in condizioni di soggetto noto.
- Per suddividere il dataset è stata utilizzata la funzione `train_test_split()` della libreria `sklearn`.
- Come prevedibile, in questo scenario l'accuratezza risulta significativamente più elevata, poiché la validazione del modello avviene su dati appartenenti a soggetti già visti nel set di addestramento.

Split 80/20 (cont.)

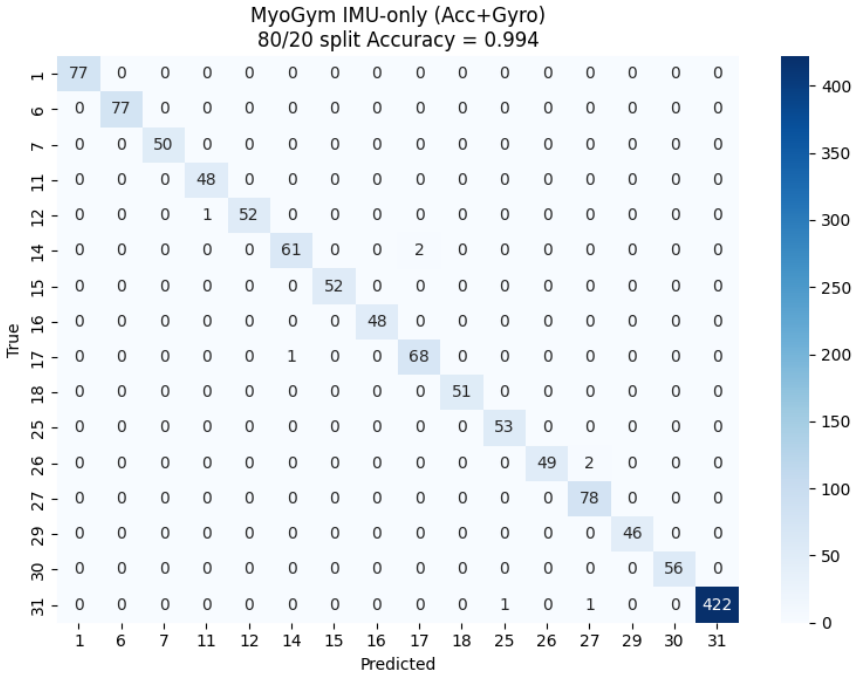

Figura 3: Matrice di confusione split 80/20

```
accuracy = 0.99
```

Feature Extraction

- I segnali IMU (accelerometro e giroscopio) sono stati segmentati tramite finestramento temporale con finestra di 4 s e passo di 1 s.
- Per ciascuna finestra sono state estratte tutte le feature in accordo con quanto riportato nel paper di riferimento.
- È stata implementata una funzione dedicata per il calcolo di tutte e 57 feature, successivamente confrontate con quelle fornite nel dataset.
- Il confronto ha mostrato elevata coerenza tra feature ricalcolate e feature originali, ad eccezione dei parametri di `Hjorth` ed `entropy`, successivamente esclusi.

Feature Extraction (cont.)

#	Prov.	Comp.	Diff.
1	0.2626	0.0047	-0.2579
2	0.1013	0.0063	-0.0950
3	0.6574	0.0124	-0.6450
4	0.0684	0.0684	0
5	0.0795	0.0795	0
6	0.1112	0.1112	0
7	0.2443	0.2443	0
8	0.9865	0.9865	0
9	-0.0201	-0.0201	0
10	0.0029	0.0029	0
11	0.5273	0.5273	0
12	-0.5366	-0.5366	0
13	0.5977	0.5977	0
14	1.3472	1.3472	0
15	0.3618	0.3618	0
16	0.2473	0.2473	0
17	0.9858	0.9858	0
18	-0.0168	-0.0168	0
19	0.2190	0.2194	0.0004
20	0.9626	0.9628	0.0001
21	-0.0566	-0.0557	0.0010
22	0.2729	0.2729	0
23	1.0120	1.0118	-0.0001
24	0.0122	0.0122	0
25	0.1667	0.1779	0.0111
26	0.9321	0.9321	0
27	-0.1497	-0.1491	0.0006
28	0.3145	0.3125	-0.0020
29	1.0574	1.0568	-0.0006

#	Prov.	Comp.	Diff
30	0.1067	0.1042	-0.0025
31	0.3474	0.3318	-0.0156
32	1.0920	1.0896	-0.0024
33	0.1780	0.1633	-0.0147
34	0.1177	0.1177	0
35	0.9089	0.9092	0.0002
36	-0.1863	-0.1834	0.0029
37	0.0370	0.0370	0
38	-0.4463	-0.4463	0
39	-0.3292	-0.3292	0
40	0	0	0
41	0	0	0
42	0	0	0
43	29	29	0
44	30	30	0
45	19	19	0
46	13.8945	13.8945	0
47	14.0544	14.0544	0
48	2.5827	2.5827	0
49	14.4482	14.4482	0
50	13.7287	13.7287	0
51	6.5313	6.5313	0
52	0.3709	0.3709	0
53	-0.1759	-0.1759	0
54	8.4348	8.4348	0
55	5.2262	4.8742	-0.3521
56	3.7244	4.5067	0.7823
57	2.3898	4.8754	2.4856

Tabella 2: Confronto tra feature `acc` fornite e calcolate

Terzo addestramento

- Le feature ricalcolate sono state utilizzate per riaddestrare il modello, al fine di verificare la coerenza dei risultati rispetto a quelli ottenuti con le feature fornite.
- Le feature associate ai parametri di `Hjorth` e `entropy` sono state eliminate per garantire un confronto equo; il set finale include **51** feature per l'accelerometro e **51** per il giroscopio.
- L'addestramento e la validazione sono stati effettuati sia con tecnica `LOSO` sia tramite suddivisione del dataset secondo lo split 80/20.
- I risultati in Tabella 3 mostrano come le differenze di prestazioni siano contenute; l'implementazione del calcolo delle feature risulta, dunque, corretto.

Terzo addestramento (cont.)

#	Provided	Computed
`LOSO`	0.911	0.895
split 80/20	0.994	0.982

Tabella 3: Accuracy con feature calcolate e fornite (senza `Hjorth` e `entropy`)

#	Subject	n_{test}	Accuracy
0	1	722	0.653259
1	6	667	0.772114
2	3	567	0.892416
3	7	656	0.905488
4	10	501	0.932136
5	5	674	0.959941
6	8	588	0.962585
7	2	817	0.963280
8	4	512	0.964844
9	9	773	0.967658

(a) analisi per soggetto (`LOSO`) con feature calcolate

#	Subject	n_{test}	Accuracy
0	1	722	0.657895
1	6	667	0.797601
2	7	656	0.910061
3	10	501	0.934132
4	3	567	0.936508
5	5	674	0.974777
6	2	817	0.975520
7	8	589	0.977929
8	9	773	0.987063
9	4	512	0.992188

(b) analisi per soggetto (`LOSO`) con feature fornite

Confronto modelli

- Sono stati confrontati diversi classificatori tramite il tool `Classification Learner` di MATLAB.
- Il confronto è stato effettuato utilizzando tutte le feature fornite (**57** acc e **57** gyro) con la medesima suddivisione dei dati (80/20).
- I modelli valutati includono le versioni ottimizzate di:
  - k-Nearest Neighbors
  - Decision Tree
  - Support Vector Machine (SVM)
  - Naive Bayes
- Il classificatore SVM ha mostrato il miglior compromesso tra accuratezza e complessità computazionale.

Confronto modelli (cont.)

Classifier	Accuracy (%)	Inference Cost	Training Time (s)
Naive Bayes (opt.)	91.7	$O(n \cdot d)$	936.98
Decision Tree (opt.)	96.6	$O(h)$	45.221
k-NN (opt.)	99.1	$O(n \cdot d)$	68.046
Linear SVM	99.3	$O(M \cdot d)$	6.4298
Gaussian SVM	99.4	$O(M \cdot n_{sv} \cdot d)$	11.287
SVM (opt.)	99.5	$O(M \cdot d)$	3079

Tabella 5: Confronto tra modelli

Con:

- d : numero di feature (nel nostro caso 114).
- n : numero di campioni di training (nel nostro caso 5183).
- C : numero di classi (nel nostro caso 16).
- S : numero di split (nel nostro caso 352).
- h : profondità dell'albero ($\approx log_2 S$).
- $M = \frac{C(C-1)}{2}$: numero di sottomodelli binari (nel nostro caso 120).
- n_{sv} : numero di support vectors (nel nostro caso 8776).

Analisi Feature con `ANOVA`

- È stata applicata una riduzione delle feature tramite analisi della varianza (`ANOVA`) per valutare la rilevanza statistica delle singole feature rispetto alle classi.
- Le feature sono state selezionate in base al valore di `F-score`, mantenendo quelle maggiormente discriminanti.
- La riduzione consente di diminuire la dimensionalità del problema, migliorando l'efficienza computazionale del modello.
- Le prestazioni del classificatore restano comparabili, confermando la presenza di ridondanza nel set di feature originale.

Analisi Feature con ANOVA (cont.)

- Quasi tutte le feature possiedono `F-score = Inf` ad eccezione:

n_{feature}	**Feature (gyro)**	ANOVA
74	`median(y)`	660.9583
109	`sum(X(6:10))`	604.2094
75	`median(z)`	535.4436
113	`entropy(y)`	500.5142
95	`corr(x,z)`	460.0047
108	`sum(Z(1:5))`	416.1677
112	`entropy(x)`	390.0310
99	`sum((z(2:end)`$\geq$`0) & (x(1:end-1)`$\leq$`0))`	372.9664
105	`sum(Z(1))`	162.6539
104	`sum(Y(1))`	93.9726
103	`sum(X(1))`	83.2138
73	`median(x)`	46.1629
64	`mean(x)`	19.4763
66	`mean(z)`	7.3320
65	`mean(y)`	0.7020

Tabella 6: Features con `F-score` diverso da `Inf`

Riduzione Feature con ANOVA

Number of Feature	**Accuracy (%)**
114/114	99.4
108/114	99.4
105/114	99.4
99/114	99.4
85/114	99.4
70/114	99.4
64/114	99.4
60/114	99.3
57/114	99.2
28/114	85.5

Tabella 7: Prestazioni del modello Linear SVM dopo riduzione delle features